Phase 2: WebSockets & a Resilient Binance Client

What we built

- **Live prices**: the watchlist and the middle panel now show **real Binance prices** (the mock data file is gone)
- `BinanceSocket`: a WebSocket client with **heartbeat**, **exponential-backoff reconnect** (with jitter), **endpoint rotation** and **resubscription**
- `useLiveTickers`: a **custom hook** that connects the socket to React state
- **Connection badge** in the header (Connecting / Live / Reconnecting / Offline) + a **"Simulate drop"** button to watch the reconnect happen
- The browser tab title shows the live price of the selected coin

Files and what each one does

File	Role
`src/config.ts`	The watchlist coins + the Binance WebSocket URLs
`src/services/binanceMessages.ts`	Understands Binance's raw JSON (`{"e":"24hrTicker","s":"BTCUSDT","c":"67250.5",...}`) and converts it to our `Ticker`
`src/services/BinanceSocket.ts`	**The core.** Plain TypeScript class (no React): connect, subscribe, heartbeat, backoff, resubscribe
`src/hooks/useLiveTickers.ts`	Custom hook: creates the socket in `useEffect`, stores prices in state, closes the socket in the cleanup
`src/components/ConnectionBadge.tsx`	The status pill + the demo button

Data flow

```
Binance server
   │  (WebSocket push, ~1 msg/sec per symbol)
   ▼
BinanceSocket.handleMessage  → JSON.parse → drop control replies
   ▼
useLiveTickers onMessage     → isTickerMessage? → toTicker() → setTickers(...)
   ▼
App re-renders → Watchlist / SymbolSummary get new props → screen updates
```

Design choice: the socket logic is a **plain class**, separate from React. It's easier to reason about and to test, and it could be reused with any UI library. The hook is just a thin "adapter" between the two.

Concepts (theory only)

1. HTTP vs WebSocket

	HTTP (REST)	WebSocket
Pattern	Request → response, then done	One connection that **stays open**
Who talks?	Only the client asks	**Both sides**, any time (full-duplex)
Getting live prices	**Polling**: ask every second (wasteful, laggy, rate-limited)	Server **pushes** each change instantly
Overhead	Headers on every request	Tiny frames after the first handshake

How a WebSocket starts: it begins as an ordinary HTTP request with an `Upgrade: websocket` header. The server answers `101 Switching Protocols`, and from then on the same TCP connection carries WebSocket frames. `wss://` = WebSocket over TLS (like `https://`).

2. The four events

`open` (connected) · `message` (data arrived) · `error` (something failed, and is always followed by close) · `close` (connection ended, with a code). `readyState`: CONNECTING(0) → OPEN(1) → CLOSING(2) → CLOSED(3). You can only `send()` while it's OPEN.

3. Binance streams & subscribe

- Endpoint `wss://stream.binance.com:9443/ws`, public with no API key.
- We send `{"method":"SUBSCRIBE","params":["btcusdt@ticker","ethusdt@ticker"],"id":1}`
- The server replies `{"result":null,"id":1}` (a control reply, which we ignore), then pushes `24hrTicker` messages about once per second per symbol.
- Numbers arrive as **strings** (`"67250.50"`) to avoid floating-point precision loss, so we convert them with `Number()`.
- Binance cuts every connection after **24 hours**, so a reconnect strategy is **required**, not optional.

4. Heartbeat (ping/pong)

Problem: a connection can be **silently dead**. The laptop sleeps or the router drops the line, and no `close` event fires for minutes. The UI shows **stale prices** as if they were live, which is dangerous in trading.

Solution: check regularly that something is still arriving. - Binance sends protocol-level ping frames and the browser answers them **automatically**, but browser JavaScript **cannot see or send** those frames. - So we built an **application-level heartbeat**. Every 5s we check the time since the last message: - silent > 10s → send `LIST_SUBSCRIPTIONS` as a "ping" (the reply is our "pong") - silent > 20s → declare it dead → drop it → reconnect

5. Exponential backoff with jitter

Retrying instantly in a loop hammers a server that's already struggling and drains the battery.
Exponential backoff: the wait doubles after each failure, 1s → 2s → 4s → 8s → 16s → 30s (capped).

Jitter: use a random delay between 50% and 100% of that wait. *Why:* when a server restarts, **thousands of clients disconnect at the same moment**. Without jitter they'd all reconnect at the same instant again (the **thundering herd**) and knock it over. Randomness spreads them out. After a **successful** connection, the attempt counter resets to 0.

6. Resubscription

A new WebSocket connection is a **blank slate**: the server has forgotten what we subscribed to. So the client keeps a `Set` of the streams it wants, and **re-sends SUBSCRIBE in `onopen`** every time it connects. Bonus: `subscribe()` called *before* the socket opens just records the streams, and `onopen` sends them. It's the same code path.

7. Endpoint rotation

On each reconnect we try the next URL (`stream.binance.com` → `data-stream.binance.vision`, Binance's market-data mirror). This helps when one domain is blocked or down.

8. Custom hooks

A function named `useSomething` that uses other hooks inside. It **packages stateful logic** so components stay simple: `const { tickers, status } = useLiveTickers(SYMBOLS)` and the component knows nothing about sockets. Every component that calls a hook gets its **own** copy of that state.

9. `useEffect` cleanup

The function returned from an effect runs when the component **unmounts** or **before the effect re-runs**. We use it to `socket.close()`. Without it: **leaked sockets**, duplicate subscriptions, and updates to unmounted components. **StrictMode** (dev only) deliberately mounts → unmounts → mounts again to expose missing cleanups. You may see a harmless console warning "WebSocket is closed before the connection is established" because of this.

10. `useRef`

Holds a mutable value that **survives re-renders** but **doesn't trigger** one when it changes. That's perfect for the socket instance (and DOM elements, in Phase 4).

11. Immutable state updates

`setTickers(prev => ({ ...prev, [symbol]: ticker }))` - **Functional update** `prev => ...`: always works from the latest state, even with rapid updates. - **New object** (`{...prev}`): React compares objects by **reference**. Changing the old object in place would look like "nothing changed".

12. Type guards & `unknown`

Data from the network is **untrusted**, so it's typed `unknown`. `isTickerMessage(msg): msg is RawTickerMessage` checks it at runtime and tells TypeScript the type afterwards.

The deliberate problem (fixed in Phase 3)

Every message → `setTickers` → **a full App re-render**. 8 symbols ≈ 8 renders/sec, which is fine. But 100+ updates/sec (many symbols, an order book, trades) means 100+ renders/sec, which is **more than the 60 frames the screen can even show**, and the UI stutters. Phase 3 fixes it with `requestAnimationFrame` batching + a Zustand store.

Try it yourself

1. Watch the prices update every second, and the tab title changing.
2. Click **Simulate drop**: the badge turns yellow ("Reconnecting · attempt 1 · retry in 0.7s"), then green again, and the prices keep flowing. That's **reconnect + resubscribe** working.
3. Turn your **Wi-Fi off** for 30s: watch the attempts grow and the delays double (backoff). Turn it back on and it recovers by itself.
4. DevTools → **Network** tab → filter **WS** → click the connection → **Messages**: see the SUBSCRIBE you sent and the raw ticker JSON arriving.

Resume mapping

> "Engineered a resilient **WebSocket client** with **heartbeat**, **exponential-backoff reconnect** and **symbol resubscription**"

All three are in `BinanceSocket.ts`. "Streaming ticks from the Binance WebSocket API" is `useLiveTickers` + `binanceMessages.ts`.

Interview Qs

1. **Why WebSocket over REST polling for prices?** The server pushes updates instantly over one persistent connection. Polling is laggy, wastes requests and hits rate limits.
2. **How does your heartbeat work?** Browsers can't send protocol pings, so I track the time since the last message. After 10s of silence I send a lightweight request as a ping. After 20s I treat the socket as dead and reconnect. This catches half-open connections that never fire `close`.
3. **Why exponential backoff? Why jitter?** Backoff avoids hammering a failing server. Jitter stops all clients reconnecting in sync after an outage (the thundering herd).
4. **What happens to subscriptions after a reconnect?** The server forgets them. I keep the desired streams in a Set and re-send SUBSCRIBE in `onopen`.
5. **How do you avoid leaking sockets in React?** I create the socket in `useEffect` and close it in the cleanup function. StrictMode double-mounting in dev helps catch mistakes.
6. **Why keep the socket in a class and not in the component?** Separation of concerns: the networking logic is testable and UI-agnostic, and the hook just adapts it to React state.
7. **What's the difference between `close()` by the user and a network close?** A `closedByUser` flag. A user close stops for good, and any other close triggers a reconnect.